

Salesforce - Microsoft Teams Case Swarm Integration

Architecture & Component Reference

Generated: July 21, 2026

Project: SFMSTeamsInOutProject

1. Executive Overview

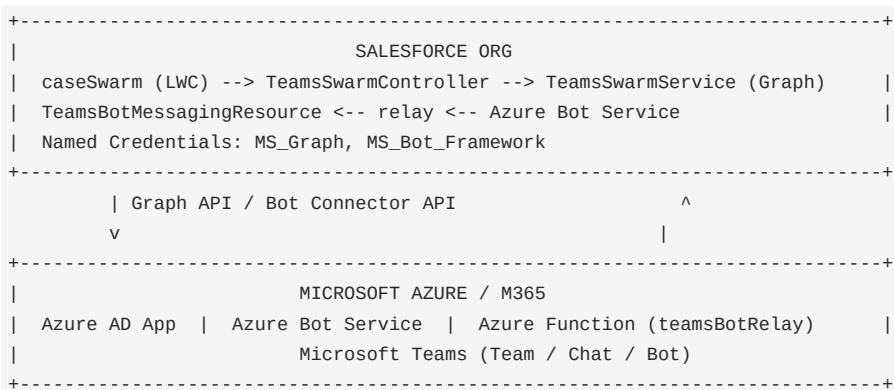
This solution enables Salesforce Service Cloud agents to start a Microsoft Teams swarm directly from a Case record. A swarm brings subject-matter experts (SMEs) into a shared Teams workspace - either a Team (M365 group) or a group Chat - so they can collaborate on resolving the case.

For Chat swarms, a Teams bot posts an interactive Adaptive Card that lets participants view and edit Case fields (Status, Priority, Owner, Subject, Description) inline inside Teams, with changes written back to Salesforce.

Key design principle: all Microsoft API calls run server-side in Apex. The Lightning Web Component (LWC) never calls Microsoft Graph or Bot Framework directly.

2. High-Level Architecture

The integration spans three platforms: Salesforce (business logic and data), Microsoft 365 / Teams (collaboration), and Azure (bot registration and HTTP relay).



3. Why Server-Side Integration?

- Authentication: Graph requires OAuth bearer tokens. Client secrets cannot safely live in browser code (LWC).
- CORS: graph.microsoft.com does not accept arbitrary browser-origin requests for app-only auth.
- Pattern: LWC -> Apex Controller -> Apex Service -> Named Credential -> Microsoft APIs.

4. Component Inventory

4.1 Salesforce - User Interface

Component	Type	Purpose
caseSwarm	LWC	Case page UI: SME search, swarm type (Team/Chat), Start swarm, Add members, Open in Teams, status polling.

4.2 Salesforce - Apex Controllers & Services

Component	Purpose
TeamsSwarmController	LWC entry point: searchUsers(), startSwarm(), addMembersToSwarm().
TeamsSwarmService	Graph API: create group, enable team, add members, create chat, install bot.
TeamsSwarmProvisionQueueable	Retries team enablement until Graph propagates new group.
TeamsSwarmProvisionScheduler	Schedules first provisioning retry.

Component	Purpose
TeamsSwarmBotInstallQueueable	Installs bot into chat asynchronously after swarm creation.
TeamsBotMessagingResource	REST /teamsbot/messages - handles Bot Framework activities.
TeamsBotJwtValidator	Validates Bot Framework RS256 JWT against JWKS.
TeamsAdaptiveCardBuilder	Builds view/edit Adaptive Card JSON (Action.Execute).
TeamsBotConversationService	Posts cards via Bot Framework Connector API v3.
TeamsBotInstalledCardQueueable	Posts initial case card after bot installation.

4.3 Salesforce - Configuration & Data

Resource	Purpose
MS_Graph / MS_Graph_Cred	Named Credential for Microsoft Graph (client credentials).
MS_Bot_Framework / MS_Bot_Framework_Cred	Named Credential for Bot Framework Connector API.
Teams_Bot_Config__mdt	Microsoft App Id and Teams App Catalog Id.
Teams_Swarm_User	Permission set for swarm Apex access.
Case custom fields	Swarm_Team_Id__c, Swarm_Group_Id__c, Swarm_Team_Url__c, Swarm_Status__c, Swarm_Type__c, Swarm_Bot_Service_Url__c, Swarm_Bot_Install_Error__c.
User custom fields	Azure_AD_Email_Id__c (required), Azure_AD_Object_Id__c (optional).

4.4 Microsoft Azure

Component	Purpose
Azure AD App Registration	Service identity for Graph and bot. Permissions: Group.ReadWrite.All, Team.Create, Directory.Read.All, Chat.ReadWrite.All, TeamsAppInstallation.ReadWriteForChat.All.
Azure Bot Service	Registers bot; messaging endpoint points to relay (not Salesforce directly).
Azure Function: teamsBotRelay	Rewrites Authorization header; OAuth to Salesforce; forwards request body.
Teams App Catalog	Custom app from teamsapp/manifest.json for bot install in chats.

5. Authentication & Connection Establishment

5.1 Salesforce to Microsoft Graph

1. Agent clicks Start swarm in LWC.
2. LWC calls TeamsSwarmController.startSwarm() via @AuraEnabled Apex.
3. Apex calls TeamsSwarmService using callout:MS_Graph.
4. Named Credential obtains OAuth token from login.microsoftonline.com (client credentials).
5. Scope: https://graph.microsoft.com/.default
6. Graph creates M365 group/Chat; members identified by User.Azure_AD_Email_Id__c.

5.2 Salesforce to Bot Framework Connector

1. TeamsBotMessagingResource captures conversation serviceUrl on bot install.
2. Stored on Case.Swarm_Bot_Service_Url__c.
3. TeamsBotConversationService posts to {serviceUrl}/v3/conversations/{id}/activities.
4. Authenticated via MS_Bot_Framework Named Credential.

5.3 Bot Framework to Salesforce (Inbound)

This connection requires the Azure Function relay.

1. Teams action triggers Bot Framework POST to Azure Bot messaging endpoint.
2. Azure Bot forwards to teamsBotRelay Azure Function.
3. Relay reads Bot JWT from Authorization header.
4. Relay gets Salesforce OAuth token (Connected App client credentials).
5. Relay forwards to /services/apexrest/teamsbot/messages with Authorization = Salesforce token and X-Bot-Framework-Authorization = Bot JWT.
6. TeamsBotJwtValidator validates Bot JWT; resource processes activity and returns card JSON for invoke actions.

5.4 Why Relay (Not Power Automate)

- Salesforce intercepts Authorization on apexrest and treats it as a Salesforce session - Bot JWT causes 401 before Apex runs.
- Relay moves Bot JWT to X-Bot-Framework-Authorization.
- Azure Function chosen for synchronous low-latency pass-through required by Teams invoke responses.

6. End-to-End User Flows

6.1 Start Team Swarm

1. Agent selects SMEs and swarm type Team.
2. TeamsSwarmService.createGroup() - POST /v1.0/groups.
3. Case: Swarm_Status__c = Provisioning, Swarm_Type__c = Team.
4. Async job retries PUT /groups/{id}/team until ready.
5. Members added via Graph \$batch; status becomes Active.

6.2 Start Chat Swarm (Interactive Bot Card)

1. TeamsSwarmService.createChat() - POST /v1.0/chats.
2. Case immediately Active; bot install queued async.
3. Graph installs bot; Teams sends conversationUpdate to relay.
4. Initial Adaptive Card posted; Edit/Save updates Case in Salesforce.

6.3 Add Members

1. Agent selects SMEs on active swarm Case.
2. Graph addMembers (Team) or addChatMembers (Chat).

7. Identity & Data Model

Users need Azure_AD_Email_Id__c (M365 email/UPN) to appear in SME picker and be added to swarms.

Case Field	Description
Swarm_Team_Id__c	Graph Team id or Chat id.
Swarm_Group_Id__c	M365 group id (Team swarms only).
Swarm_Team_Url__c	Deep link to open in Teams.
Swarm_Status__c	Empty, Provisioning, Active, or Failed.
Swarm_Type__c	Team or Chat.

Case Field	Description
Swarm_Bot_Service_Url__c	Bot Framework serviceUrl for posting cards.
Swarm_Bot_Install_Error__c	Diagnostic if bot install fails.

8. Security Model

- Outbound: OAuth client credentials in Salesforce External Credentials.
- Inbound: Bot Framework JWT validated (RS256, issuer, audience, expiry).
- Relay OAuth proves integration caller; Bot JWT proves Bot Framework origin.
- Case updates scoped: Swarm_Team_Id__c must match conversation id.

9. Deployment Checklist

Step	Platform	Action
1	Azure AD	Register app; Graph permissions; client secret.
2	Salesforce	MS_Graph Named Credential.
3	Salesforce	Case/User fields; populate Azure_AD_Email_Id__c.
4	Salesforce	Deploy package; add caseSwarm LWC to Case page.
5	Azure	Create Azure Bot; messaging endpoint = relay URL.
6	Azure	Deploy teamsBotRelay Function App.
7	Salesforce	Connected App for relay OAuth.
8	Teams	Upload teamsapp manifest; note catalog id.
9	Salesforce	Teams_Bot_Config__mdt; MS_Bot_Framework credential.
10	Salesforce	Expose REST endpoint for relay.

10. Known Limitations

- Interactive bot card works for Chat swarms only, not Team/channel swarms.
- Team provisioning is asynchronous (Provisioning status until Graph completes).
- JWKS fetched on every bot request (fine at low volume).
- Relay uses real integration user OAuth instead of Guest Site (Case access restrictions).

- End of document -